

Progress Update 2 — 05 May

Project: OptiTrade — Financial trading dashboard (dynamic canvas, widgets, and AI-assisted insights)

1.0 Introduction

This report summarizes Progress Update 2 for OptiTrade, focusing on implementation outcomes, preliminary technical results, key challenges, mitigation plans, and the updated delivery roadmap.

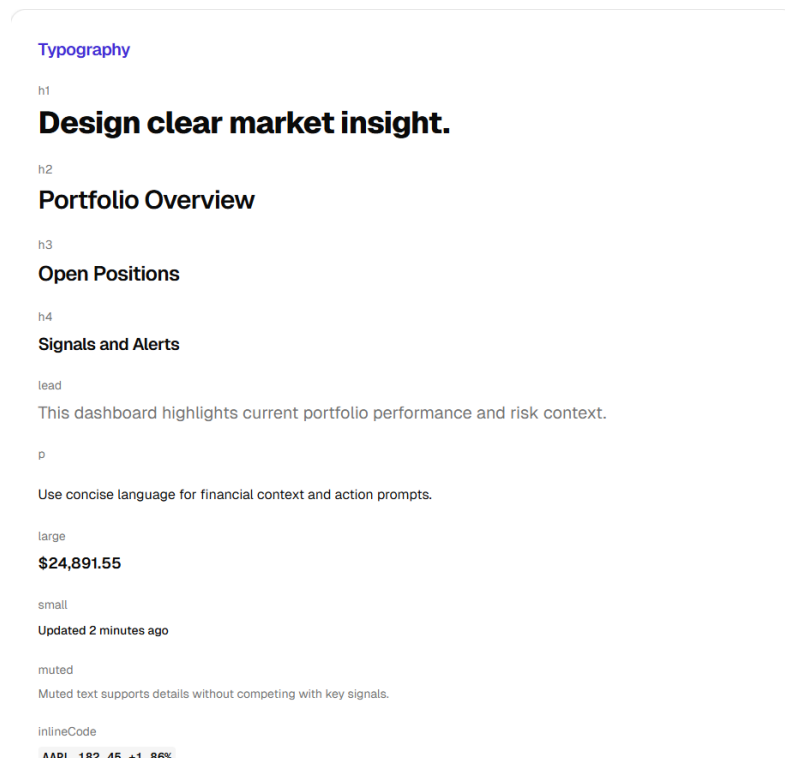
2.0 Work completed

Deliverable	Status	Completed / notes
Dynamic canvas enhancements	Completed	Implemented responsive grid recalculation, drag preview, and collision handling.
Chart widget (symbols/indicators/API)	Completed	Added multi-symbol support, three indicators, and FMP data integration.
News widget (LLM + pipeline)	Completed	Integrated local Llama model, sentiment/risk analysis, and <code>`/api/news`</code> .
Portfolio widget + backend APIs	Completed	Implemented UI, live/demo fallback, and portfolio API contract tests.
Context-aware AI chat panel	In progress	Frontend/backend redesign completed; OpenUI and context-passing refinement planned.

3.0 Preliminary results

3.1 Design principles

The project design direction has shifted toward a more disciplined and consistent visual system. A centralized typography and color setup in ``theme.stories.tsx`` now standardizes visual behavior across widgets. Components are rendered in a controlled canvas with explicit size variants, resulting in a more coherent and natural dashboard experience.



A storyboard with typography variants was prepared to validate visual consistency before implementation.

3.2 Dynamic canvas

To improve consistency and usability, the dynamic canvas was enhanced as follows:

- Static dimensions: Cell units are defined in pixels to maintain layout consistency across viewport sizes.
- Responsive layout calculation: The number of grid columns is recalculated using `ResizeObserver` and container width.
- Drag preview: While widgets are dragged, a live and precise overlay shows the predicted drop target.
- Placement normalization and resolution: When collisions occur, a fallback search algorithm finds the next valid, collision-free slot.
- Animation update: Enter/exit transitions for edit mode and drag-source handoff into the canvas were smoothed.

3.3 Price and technical indicator chart widget

Compared with previous updates, the chart widget has been enhanced in the following areas:

1. Charting Experience

Symbols Selection Experience

The charting workflow now supports faster and more flexible analysis. Users can select up to three symbols in one view, remove symbols via the close icon, and add new symbols by entering a ticker. This improves comparison while keeping interactions simple.

Three major indicators

Three technical indicators were added: Moving Average, Bollinger Bands, and RSI. Users can toggle each indicator on or off, and the corresponding lines update on the chart. This adds analytical control without increasing complexity.

2. Stock Chart API Integration

FMP API Integration

The widget now retrieves historical price data in near real time through Financial Modeling Prep (FMP) stock chart API integration. This keeps chart data current and accurate for selected symbols.

Charting Library

The TradingView library is used as the charting engine to provide a robust, interactive experience. Features such as crosshair inspection allow users to review historical prices with higher precision.

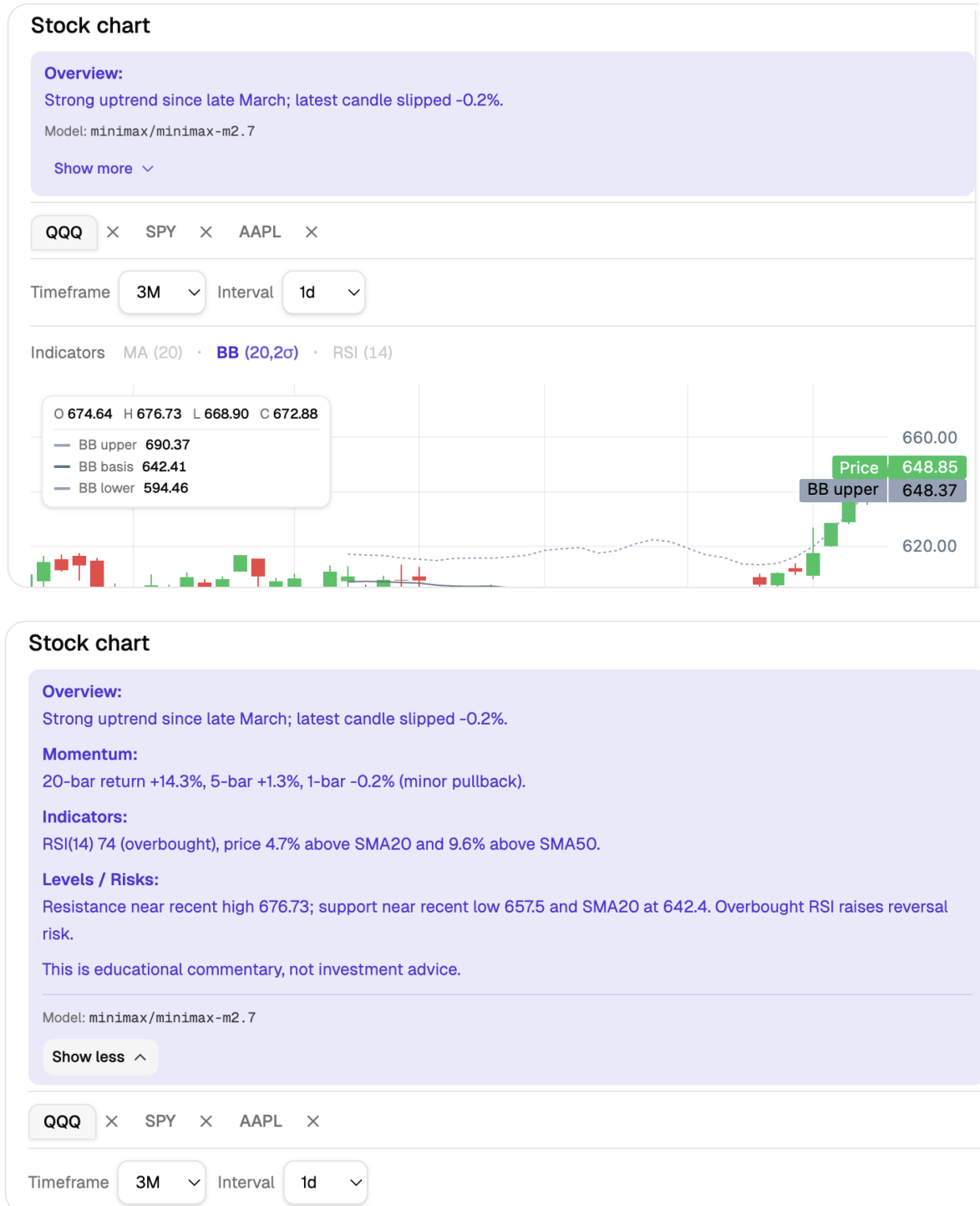
3. AI Intelligence Summary

AI summary inference experience

An AI summary module was added to help users interpret chart activity more efficiently. When the chart updates, the widget triggers inference and generates a structured response based on historical prices and selected technical indicators.

Prompting Approach

A structured few-shot prompting framework was introduced to guide model output. This improves consistency, relevance, and readability.



3.4 News widget

1. AI model integration: Llama 3.2 3B

- Local LLM deployment

The Ollama framework was integrated successfully with the local `llama3.2:3b`

model. This avoids external API dependency, improves data privacy, and reduces operating cost.

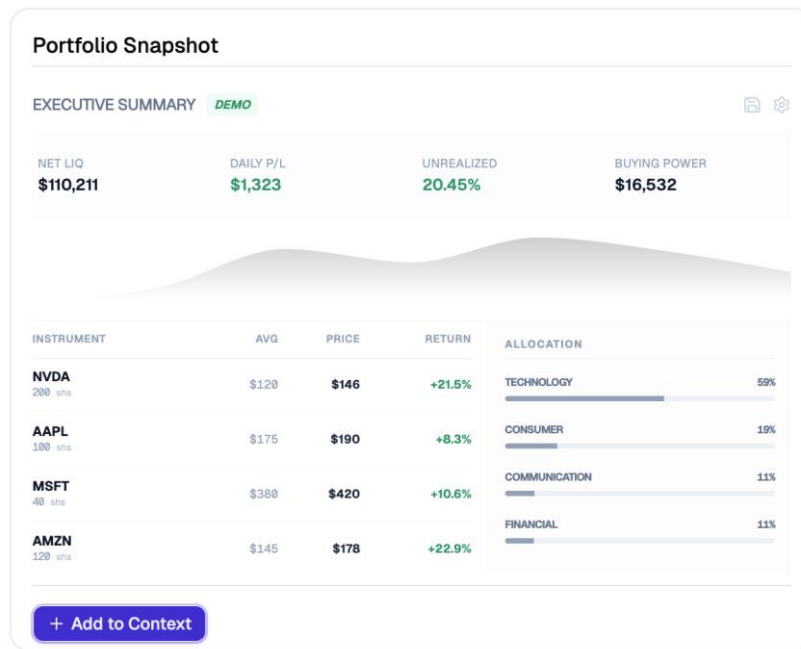
- **Sentiment analysis**
The model outputs a sentiment score from -1.0 to +1.0 for each article and automatically classifies sentiment as Positive, Neutral, or Negative.
- **Risk tag generation**
The AI automatically assigns risk labels ("High Risk", "Medium Risk", "Low Risk") to help users quickly identify major events.
- **Reasoning explanation**
Each news article includes an AI-generated rationale explaining the sentiment score and risk assessment, improving transparency and trust.
- **Fallback mechanism**
The system automatically switches to keyword-based mock analysis when the LLM service is unavailable, ensuring continuous operation.

2. News aggregation API

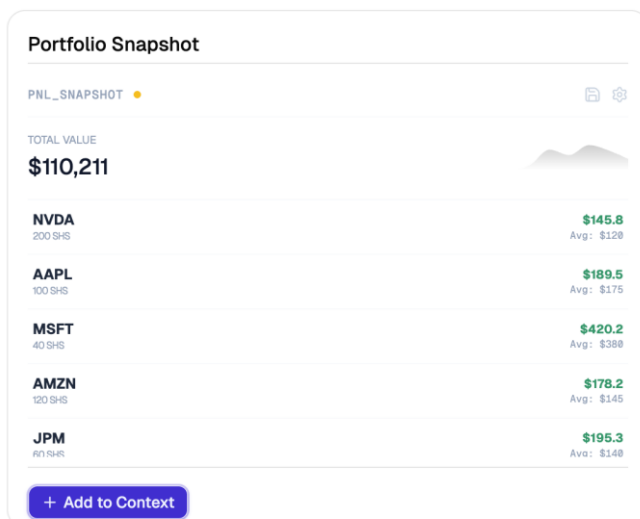
- **Multi-source news aggregation**
Two news sources were integrated successfully:
 - **Yahoo Finance RSS** – International financial news
 - **The Economic Times RSS** – Asian and emerging market news
- **Automated pipeline**
A complete news processing pipeline was implemented:
 - Scheduled fetching from RSS sources
 - Automatic filtering of non-financial content
 - Title-based deduplication to avoid redundant news
- **AI analysis integration**
Fetched news is sent automatically to the Llama model for sentiment and risk analysis, producing structured JSON output.
- **API endpoint development**
A `GET /api/news` endpoint was built for frontend widget consumption, supporting:
 - Real-time access to latest analysis results
 - Caching mechanism to reduce redundant computation

3.5 Portfolio widget

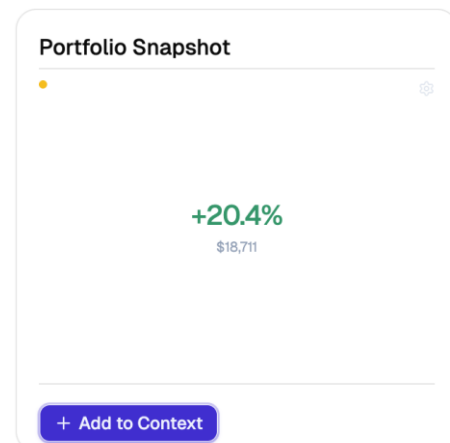
The portfolio widget has been implemented with both frontend and backend support. On the frontend, it now displays holdings, total portfolio value, unrealized P/L, daily P/L, intraday history, and sector allocation. It supports small, medium, and large layouts for different dashboard canvas sizes.



Large



Medium



Small

The widget is connected to the backend through `GET /api/portfolio``. When the backend is running, it loads live data and shows a "Live" status indicator. If the backend is unavailable, it falls back to demo data so the dashboard remains usable during development.

The backend portfolio API provides a structured snapshot containing positions, summary metrics, allocation data, and chart history. A `POST /api/paper-portfolio`` endpoint was added to save paper portfolio records with local JSON persistence as a lightweight placeholder for future database integration. The IBKR connection panel is connected to `POST /api/portfolio/connect``, allowing broker connection settings to be submitted from the widget UI.

Tests were added for the portfolio API contract, including portfolio retrieval, paper portfolio creation, broker connection validation, and route validation. Frontend linting and TypeScript checks also pass for the updated widget.

Next steps

- Implement "Add to Context" Logic: Develop the frontend functionality to capture real-time portfolio snapshots—including positions, sector allocation, and P/L—and send them as structured JSON to the AI chat panel.
- Design Portfolio-Specific Prompts: Create specialized system prompts that allow the AI to generate "Reasoning Explanations" regarding portfolio risk and performance metrics.
- State Management & UI Feedback: Add visual indicators to the chat panel to show when portfolio data is successfully "In Context," ensuring the user knows the AI is referencing their specific holdings.
- Fallback Strategy: Ensure that if the local LLM service is unavailable, the "Add to Context" feature gracefully falls back to basic rule-based analysis or status alerts.
- Security Validation: Verify that the integration maintains data privacy by keeping all "Add to Context" data within the local infrastructure.

3.6 Context-aware AI chat panel

The chat panel was redesigned across both frontend and backend.

Frontend:

- A custom Shadcn/UI chat panel replaced the Dify floating bubble.

Backend:

- A custom WebSocket connection was implemented for the Nanobot AI agent hosted on a cloud server.
- A custom system prompt (`SOUL.md`), tools, and skills were configured for the Nanobot AI agent.

Next steps:

- Investigate integration of OpenUI (<https://github.com/thesysdev/openui>) into the chat panel.
- Investigate how the "Add context" button should pass context data into the chat panel.
- Draft a question-and-expected-answer dataset for evaluation using both an LLM judge and a human expert judge.

3.7 Project web page

After obtaining access to the WordPress admin dashboard, we began migrating the Phase 1 webpage to the departmental website. Integration is still in progress due to theme customization constraints.

4.0 Challenges encountered and mitigation plan

4.1 Administrative / process

The cost claiming process is still uncertain. It may hinder the technological choice and later deployment work. Mitigation: actively engage with department and seek help from program director.

5.0 Updated plan and next steps

Phase	Focus	Duration
Development	Frontend and Backend Performance Fine-tuning	1.5 Weeks
Development	AI Performance Evaluation and Fine-tuning	1.5 Weeks
Deployment	Infrastructure Setup and Deployment	2 Weeks
Test	Testing (unit, integration, UAT) and performance	2 Weeks